FORGE

Full Project Context Document

Agentic Dev Studio — V1 Complete, V2 In Design

Owner: Suraj Doifode | GitHub: github.com/SurajD45/forge-p1

This document is the authoritative context source for any AI tool, collaborator, or developer working on FORGE.

HOW TO USE THIS DOCUMENT

If you are an AI tool: read sections 1-6 before answering any code question about FORGE.
If you are a new collaborator: read sections 1-4 first, then section 7 for your branch rules.
If you are Suraj resuming work: start from section 8 — Current State and Next Steps.
Every design decision in this document was made deliberately. Do not change architecture without reading section 5 first.

01 Project Identity and Purpose

Property	Value
Project Name	FORGE — Agentic Dev Studio
Internal Code	P1
Repository	github.com/SurajD45/forge-p1
Owner	Suraj Doifode
Current Version	V1 — Complete and Stable
Next Version	V2 — In Design Phase
Primary Language	Python 3.11
Agent Framework	CrewAI
LLM Provider	Groq (llama-3.3-70b-versatile + moonshotai/kimi-k2-instruct)

What FORGE Does

FORGE is a multi-agent SDLC pipeline. A user describes a backend project idea in natural language. FORGE runs four AI agents in sequence and produces a complete, validated, reviewed FastAPI backend project — including all Python source files, a README, and a review report listing any remaining issues.

What FORGE Does NOT Do

- It does not execute the generated code (no sandboxing in V1)
- It does not fix bugs automatically (Reviewer reports only — auto-correction is V2)
- It does not support frameworks other than FastAPI (V1 locked)
- It does not support databases other than SQLite and PostgreSQL (V1 locked)
- It does not have a web interface (V1 is CLI only — V2 adds API layer)

02 Core Philosophy — The Most Important Section

THE RULE THAT GOVERNS EVERY DECISION IN THIS PROJECT

LLM handles reasoning: understanding natural language, extracting intent, generating code.
Deterministic code handles control: making decisions, validating output, routing the pipeline.
These two responsibilities are NEVER mixed.
If you are an AI tool helping with this project: do not suggest putting routing logic inside LLM prompts.
Do not suggest letting the LLM decide which files to generate or in what order.
All control decisions are made by Python code. Always.

Why This Philosophy

LLMs are stochastic — the same input can produce different output. If control decisions are made by the LLM, the pipeline behaves differently on every run. A deterministic control plane means the pipeline structure is predictable and testable, even when the LLM output varies.

Failure Philosophy

FORGE fails loudly, never silently. If a validation check fails, the pipeline halts immediately with a clear error message. It never silently uses a fallback, never auto-corrects without logging, and never writes partial output to disk. All-or-nothing atomicity is enforced at every stage.

03 V1 Architecture — Four Agents, Four Stages

Pipeline Flow

```
python main.py
  └── orchestrator.run_pipeline()
        ├── Stage 1: Explorer Agent
        │     ├── LLM: iterative discovery interview (max 2 rounds)
        │     ├── DETERMINISTIC: auth signal correction, feature cap (max 6)
        │     ├── DETERMINISTIC: TRD builder (scale→DB, needs_auth→auth)
        │     ├── DETERMINISTIC: jsonschema validation
        │     └── OUTPUT: output/TRD.json + output/TRD.md
        │
        ├── Stage 2: Architect Agent
        │     ├── LLM: file structure + module responsibilities
        │     ├── DETERMINISTIC: mandatory file injection (auth.py if jwt, alembic
if pg)
        │     ├── DETERMINISTIC: NON_CODE_FILES stripping (requirements.txt etc)
        │     ├── DETERMINISTIC: jsonschema validation
        │     └── OUTPUT: output/ARCH.json + output/ARCH.md
        │
        ├── Stage 3: Developer Agent
        │     ├── config.py generated FIRST (bootstrap — everything depends on it)
        │     ├── Remaining files in dependency_order from ARCH.json
        │     ├── Per file: LLM call with TRD + ARCH + all previous files as
context
        │     ├── Per file: ast.parse() validation — retry up to 2x with error
message
        │     ├── All files held in memory until ALL pass validation
        │     ├── Atomic write to output/generated/{project_name}/
        │     ├── LLM README generation (non-blocking, 3 retry attempts)
        │     └── OUTPUT: all .py files + README.md
        │
        └── Stage 4: Reviewer Agent
              ├── Layer 1: deterministic AST checks (7 checks, no LLM)
              ├── Layer 2: kimi-k2-instruct semantic review (non-blocking)
              └── OUTPUT: output/generated/{project_name}/REVIEW_REPORT.md
```

04 V1 File Structure — Every File Explained

File / Folder	Purpose
main.py	3-line CLI entry point. Imports orchestrator, calls run_pipeline().
pipeline/orchestrator.py	Controls all 4 stages. Rate limit retry logic. Bootstrap file ordering. Atomic write coordination.
agents/explorer.py	Explorer Agent. run_explorer() for question rounds. force_intent_extraction() when MAX_ROUNDS hit. _validate_intent_fields() for safe defaults.
agents/architect.py	Architect Agent. run_architect(trd) — LLM designs file structure from TRD.
agents/developer.py	Developer Agent. generate_file() per file with context injection. generate_readme() for LLM README. All prompt rules live here.
agents/reviewer.py	Reviewer Agent Layer 2. run_llm_review() using kimi-k2-instruct. Returns {critical, warnings, passed} JSON.
utils/llm_client.py	Key rotation utility. Reads GROQ_API_KEY through GROQ_API_KEY_5. Auto-rotates on RateLimitError.
utils/trd_builder.py	Deterministic TRD builder. Maps scale→database, needs_auth→auth. Writes TRD.json and TRD.md.
utils/arch_builder.py	Deterministic ARCH builder. Mandatory file injection. NON_CODE_FILES stripping. Writes ARCH.json and ARCH.md.
utils/ast_validator.py	ast.parse() wrapper. Returns (bool, error_message). Used by orchestrator for per-file validation.
utils/ast_checker.py	Reviewer Layer 1. 7 deterministic structural checks. Returns list of {check, file, issue, severity}.
utils/dev_builder.py	Atomic file write. get_output_dir() for Docker/local path resolution. write_generated_files(). cleanup_on_failure().
utils/review_builder.py	Generates REVIEW_REPORT.md from Layer 1 + Layer 2 results. UTF-8 encoded write.
utils/schema_validator.py	validate_trd() and validate_arch() using jsonschema.
utils/file_utils.py	Shared read_json(), write_json(), write_markdown() helpers.
schemas/trd_schema.json	Frozen TRD contract. Validation failure halts pipeline. Never modify without discussion.
schemas/arch_schema.json	Frozen ARCH contract. Same rules.
output/	All generated artifacts. .gitignored. output/.gitkeep keeps folder tracked.

output/generated/{name}/	Per-project generated files: all .py files, README.md, REVIEW_REPORT.md

05 Decision Log — Why We Made Every Major Choice

This section is critical for any AI tool or collaborator. Every decision here was made deliberately after considering alternatives. Do not suggest reverting to rejected options without understanding why they were rejected.

Decision	What we chose and why we rejected alternatives
Agent framework	CrewAI over LangChain. LangChain is more flexible but significantly more complex. CrewAI's Agent/Task/Crew abstraction is clean and appropriate for structured sequential pipelines.
LLM for Explorer+Architect+Developer	groq/llama-3.3-70b-versatile. Fast, high TPM limit on Groq. Tried qwen3-32b — decommissioned. Tried kimi-k2 for generation — too low TPM on free tier.
LLM for Reviewer	groq/moonshotai/kimi-k2-instruct. Separate model = separate rate limit. Strong code reasoning. Used only for semantic review, not generation.
Code validation	ast.parse() only. Never execute generated code — no sandboxing in V1. AST catches all syntax errors deterministically.
Output write strategy	All-or-nothing atomic write. Files held in file_buffer dict until all pass AST. Partial projects are worse than no project.
Context injection strategy	Linear injection — every file sees all previously generated files. Explicit dependency mapping is V2.
config.py generation order	config.py always generated first (bootstrap). Everything imports from config. Enforced by BOOTSTRAP_FILES constant in orchestrator.
Rate limit handling	5-key rotation (GROQ_API_KEY through GROQ_API_KEY_5) + 20s inter-file delay + progressive backoff retry. No single key can sustain a full pipeline run on free tier.
Reviewer behavior	Non-blocking. Reports issues, does not fix them. Auto-correction loop is V2. Blocking on reviewer would halt too many valid pipelines.
Docker strategy	Single image with all dependencies. stdin_open + tty for interactive input(). Volume mount for output/ folder.
V1 scope lock	FastAPI only, SQLite/PostgreSQL only, API projects only. Scope lock prevents half-built multi-framework support. Flask is V2, CLI projects are V3.

06 Known Issues and Prompt Rules

Bugs Caught During V1 Testing

These bugs were identified during 3-test validation runs and fixed by adding explicit rules to the Developer Agent prompt in agents/developer.py. All rules are documented here so future developers understand why they exist.

Bug	Fix applied in developer.py
encodedjwt typo in auth.py	Added explicit CORRECT/WRONG example: encoded_jwt = jwt.encode(...); return encoded_jwt
HTTPBearer instead of OAuth2PasswordBearer	Banned HTTPBearer explicitly. oauth2_scheme = OAuth2PasswordBearer(tokenUrl='token') enforced.
datetime fields as str in Pydantic	DATETIME TYPE RULE: all datetime fields must use datetime type, never str.
Package-prefixed imports (from project.database import)	CORRECT: from database import get_db / WRONG: from project_name.database import get_db
Plain password storage	PASSWORD HASHING RULE with CORRECT/WRONG example. get_password_hash() mandatory.
Missing /token endpoint	Full /token endpoint example added to main.py rules. OAuth2PasswordRequestForm required.
current_user typed as wrong model	CURRENT USER TYPE ANNOTATION RULE: always User, never Product or Task.
SQLAlchemy 'and' in filter	SQLALCHEMY FILTER RULE: use comma-separated args, never Python 'and'.
DATABASE_URL defaulting to None	Fallback string added: DATABASE_URL = settings.DATABASE_URL or 'postgresql://...'
Duplicate FastAPI import in main.py	'Import FastAPI ONCE only' rule added to main.py section.
models.py using __init__ methods	Never define __init__ in SQLAlchemy models rule added.
settings.datetime usage	Never use settings.datetime — import datetime from datetime module rule added.

Persistent Issues (Not Yet Fixed)

Still appearing in Reviewer reports — being tracked

Relationship back_populates misconfiguration — complex SQLAlchemy relationship errors.

Forward reference resolution failures in nested Pydantic models (e.g. CartItemResponse references ProductResponse defined later).
Missing search/filter endpoints when TRD includes search as a feature.
str | None Python 3.10+ syntax — incompatible with older Python versions.
These will be addressed in the auto-correction loop (V2).

07 V1 Contracts — Frozen Schemas

TRD Schema (schemas/trd_schema.json)

```
{
  "project_name": string (pattern: ^[a-z0-9-]+$)
  "project_type": "api"  // always "api" in V1
  "stack": "fastapi"     // always "fastapi" in V1
  "database": "postgresql" | "sqlite"
  "auth": "jwt" | "none"
  "features": array of strings (max 6 for MVP scope)
  "constraints": array of strings
  "out_of_scope": array of strings
  "overrides": array  // reserved for future use
}
```

ARCH Schema (schemas/arch_schema.json)

```
{
  "project_name": string (pattern: ^[a-z0-9-]+$)
  "file_list": array of strings (min 1 item)
  "module_responsibilities": object (string→string map)
  "dependency_order": array of strings
  "entry_file": string
  "app_object": string  // always "app"
  "framework": "fastapi"  // always "fastapi"
  "database": "postgresql" | "sqlite"
  "auth": "jwt" | "none"
}
```

Decision Rules (Deterministic — Never Change Without Code Review)

Signal	Decision
scale == 'high' (thousands of users mentioned)	database = postgresql
scale == 'low' (default)	database = sqlite
needs_auth == true	auth = jwt
needs_auth == false (default)	auth = none
Auth keywords in features (user/login/account/member/role/permission)	needs_auth forced to true
auth == 'jwt'	auth.py injected into file_list
database == 'postgresql'	alembic.ini added to responsibilities
Any file in NON_CODE_FILES list	Stripped from file_list and module_responsibilities

08 Current State — What Is Done and What Is Not

V1 — Completed and Stable

- Explorer Agent: iterative discovery, MAX_ROUNDS=2, force extraction, auth signal correction
- Architect Agent: file structure design, mandatory file injection, NON_CODE_FILES stripping
- Developer Agent: per-file generation, AST validation with retry, atomic write, LLM README
- Reviewer Agent: Layer 1 AST checks (7 checks) + Layer 2 kimi-k2 semantic review
- 5-key rotation: GROQ_API_KEY through GROQ_API_KEY_5 with auto-failover
- Docker: working, interactive, volume mount for output/
- GitHub: github.com/SurajD45/forge-p1, protected main branch
- 3 test cases validated: blog API, task manager, e-commerce backend

NOT YET BUILT — Common Misconceptions

Read this before assuming any feature exists

Auto-correction loop: Reviewer REPORTS bugs but does NOT fix them. This is V2.
Web interface: V1 is CLI only. No frontend. No HTTP API. Just python main.py.
Multi-user support: V1 is single-user CLI. No auth, no isolation, no concurrent runs.
Cloud storage: V1 writes to local disk only. output/ folder on the machine running it.
Flask/Django support: Not built. FastAPI only in V1.
Code execution/testing: Generated code is never executed. AST validates syntax only.
Jenkins/CI pipeline: Not set up. Manual git push only.
Deployment: V1 runs locally. No live URL. No server.

09 V2 Design — What We Are Building Next

V2 Goal

Transform FORGE from a single-user CLI tool into a deployed multi-user API service. A user registers, submits a project idea via HTTP, receives a pipeline_id, polls for status, and downloads their generated project when complete.

V2 Tech Stack

Component	Technology and reason
API layer	FastAPI — same as V1, consistency
Job queue	Celery — async job processing, workers handle pipeline runs
Message broker	Upstash Redis (free hosted) — Celery needs a broker
Database	Supabase PostgreSQL — managed, free tier, built-in auth
File storage	Supabase Storage — S3-compatible, integrates with Supabase auth
Authentication	Supabase Auth with Row Level Security (RLS)
Deployment	Oracle Cloud Free Tier — 1GB RAM VM, permanently free

V2 API Endpoints

Endpoint	Purpose
POST /auth/register	Create account — handled by Supabase Auth
POST /auth/login	Get JWT token — handled by Supabase Auth
POST /submit-project	Queue pipeline job. Returns {pipeline_id, status: 'queued'} in <1 second.
GET /pipeline-status/{pipeline_id}	Returns {status, current_stage, progress}. Client polls every 3-5 seconds.
GET /pipeline-result/{pipeline_id}	Returns download URLs for all generated files from Supabase Storage.

V2 Database Schema

```
-- Supabase manages auth.users automatically

CREATE TABLE pipeline_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  user_id UUID REFERENCES auth.users(id) NOT NULL,
  project_idea TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'queued',
  -- status values: queued | stage_1_running | stage_2_running |
  --                stage_3_running | stage_4_running | completed | failed
  current_stage INTEGER DEFAULT 0,
  error_message TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE pipeline_artifacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  pipeline_run_id UUID REFERENCES pipeline_runs(id) NOT NULL,
  artifact_type TEXT NOT NULL,
  -- artifact_type values: trd_json | trd_md | arch_json | arch_md |
  --                       generated_file | readme | review_report
  file_name TEXT NOT NULL,
  storage_path TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Row Level Security
ALTER TABLE pipeline_runs ENABLE ROW LEVEL SECURITY;
ALTER TABLE pipeline_artifacts ENABLE ROW LEVEL SECURITY;

CREATE POLICY user_isolation ON pipeline_runs
  FOR ALL USING (user_id = auth.uid());

CREATE POLICY artifact_isolation ON pipeline_artifacts
  FOR ALL USING (
    pipeline_run_id IN (
      SELECT id FROM pipeline_runs WHERE user_id = auth.uid()
    )
  );
```

V2 File Isolation Strategy

```
# Storage path pattern in Supabase Storage:
# {user_id}/{pipeline_id}/{filename}
#
# Example:
# abc123/pipe_xyz/TRD.json
# abc123/pipe_xyz/ARCH.json
# abc123/pipe_xyz/generated/team-task-manager/main.py
# abc123/pipe_xyz/generated/team-task-manager/REVIEW_REPORT.md
#
# One pipeline at a time per user enforced by:
# Check DB for any pipeline_run with status NOT IN ('completed','failed')
# If found -> return 429 with message 'Pipeline already running'
```

V2 Project Structure Addition

```
forge-p1/
├── api/                          # NEW in V2
│   ├── __init__.py
│   ├── main.py                   # FastAPI app for V2 API
│   ├── dependencies.py           # Supabase client, auth middleware
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── projects.py           # submit, status, result endpoints
│   │   └── auth.py               # register, login endpoints
│   ├── models/
│   │   ├── __init__.py
│   │   └── schemas.py            # Pydantic request/response models
│   └── worker/
│       ├── __init__.py
│       ├── celery_app.py         # Celery app definition + Redis config
│       └── tasks.py              # Pipeline Celery task — wraps run_pipeline()
│
├── pipeline/                     # UNCHANGED from V1
├── agents/                       # UNCHANGED from V1
├── utils/                        # UNCHANGED from V1
├── main.py                       # UNCHANGED — CLI still works
├── docker-compose.yml            # UPDATED — adds api, worker, redis services
└── requirements.txt              # UPDATED — adds celery, redis, supabase-py
```

Auto-Correction Loop (V2 Feature)

The Reviewer Agent currently reports bugs but does not fix them. In V2, after Layer 1 and Layer 2 checks complete, the orchestrator will attempt targeted regeneration of files with critical issues.

```
# Auto-correction flow (V2):
Reviewer finds critical issue in routes.py
→ Re-call Developer Agent for routes.py only
→ Inject: original TRD + ARCH + all other generated files + the specific critical
issue
→ Regenerate routes.py
→ Re-run Layer 1 + Layer 2 on new file only
→ If issue resolved: replace file in file_buffer, rewrite to storage
→ If still failing: mark as UNRESOLVED in REVIEW_REPORT, keep original
→ Max 1 correction attempt per file (not per issue)
```

10 Environment Variables — Complete Reference

Variable	Required
GROQ_API_KEY	Yes
GROQ_API_KEY_2	No
GROQ_API_KEY_3	No
GROQ_API_KEY_4	No
GROQ_API_KEY_5	No
CREWAI_DISABLE_TELEMETRY	Yes
OTEL_SDK_DISABLED	Yes
SUPABASE_URL	V2 only
SUPABASE_ANON_KEY	V2 only
SUPABASE_SERVICE_ROLE_KEY	V2 only
REDIS_URL	V2 only

11 Rules for AI Tools Working on This Project

MANDATORY — Read before generating any code

1. The pipeline control plane is always deterministic Python code. Never put routing logic in LLM prompts.
2. Never modify schemas/trd_schema.json or schemas/arch_schema.json without explicit instruction.
3. Never change the dependency order logic in orchestrator.py without understanding BOOTSTRAP_FILES.
4. The existing V1 pipeline (pipeline/, agents/, utils/) must not be modified for V2 work. V2 wraps V1 — it does not rewrite it.
5. All imports in generated projects must be flat (from database import X, not from app.database import X).
6. Every new file must follow the existing naming conventions and folder structure.
7. Fail loudly — never add silent fallbacks or try/except blocks that swallow errors.
8. The output path logic must handle both Docker (/app/output) and local (../output) environments.

What You Can Help With

- Writing or improving code in api/ folder (V2 work)
- Writing Celery task in api/worker/tasks.py
- Writing Supabase integration in api/dependencies.py
- Improving Developer Agent prompt rules in agents/developer.py
- Adding new AST checks to utils/ast_checker.py
- Writing tests for any component

What You Must NOT Change

- schemas/trd_schema.json and schemas/arch_schema.json — frozen contracts
- Decision rules in utils/trd_builder.py — deterministic mappings
- BOOTSTRAP_FILES and INTER_FILE_DELAY constants in orchestrator.py
- The all-or-nothing atomic write pattern in utils/dev_builder.py
- The MAX_ROUNDS=2 limit in orchestrator.py